Semi-Automated Logging of Contact Center Telephone Calls

Roy J. Byrd
(Emeritus)
IBM T.J. Watson Research Center
P. O. Box 704
Yorktown Heights, NY 10598
001-914-784-6815
roybyrd@us.ibm.com

Mary S. Neff,
Wilfried Teiken, Youngja Park,
Keh-Shin F. Cheng,
Stephen C. Gates
IBM T.J. Watson Research Center
maryneff,wteiken,young_park,
kfcheng,scgates@us.ibm.com

Karthik Visweswariah
IBM India Research Lab
Bangalore, India
091-080-417-75310
v-karthik@in.ibm.com

ABSTRACT

Modern businesses use contact centers as a communication channel with users of their products and services. The largest factor in the expense of running a telephone contact center is the labor cost of its agents. IBM Research has built a new system, Contact-Center Agent Buddies (CAB), which is designed to help reduce the average handle time (AHT) for customer calls, thereby also reducing their cost. In this paper, we focus on the call logging subsystem, which helps agents reduce the time they spend documenting those calls. We built a Template CAB and a Call Logging CAB, using a pipeline consisting of audio capture of a telephone conversation, automatic speech recognition, text analysis, and log generation. We developed techniques for ASR text cleansing, including normalization of expressions and acronyms, domain terms, capitalization, and boundaries for sentences, paragraphs, and call segments. We found that simple heuristics suffice to generate high-quality logs from the normalized sentences. The pipeline yields a candidate call log which the agents can edit in less time than it takes them to generate call logs manually. Evaluation of the Call Logging CAB in an industrial contact center environment shows that it reduces the amount of time agents spend logging calls by at least 50% without compromising the quality of the resulting call documentation.

General Terms

Algorithms, Design, Experimentation, Management

Categories and Subject Descriptors

I.2.7 [**Artificial Intelligence**]: Speech Recognition and Synthesis; Text analysis; H.3.1 [**Information Storage and Retrieval**]: Content Analysis and Indexing, – *abstracting methods;* J.1 [**Administrative Data Processing**]: Business

CIKM'08, October 26–30, 2008, Napa Valley California, USA.

1. INTRODUCTION

In recent years, contact centers have grown in importance as an interface of an organization to its customers, both existing and new. A customer's experience with the call center is seen as a factor in customer retention, as well as an opportunity to sell more to existing customers (cross-sell, up-sell) and to sell to new customers.

Contact center operations are typically labor-intensive, as they most often involve a voice conversation between a customer and an agent. Recent improvements in automatic speech recognition (ASR), information retrieval, and text analytics applied to ASR call audio offer the possibility of improving the quality of call handling and reducing the cost of contact center operations. For instance, off-line analytics can flag "bad" calls (calls with an undesirable outcome, or calls where the agent falls short of call standards) for future study aimed at improvement of agent performance [8][17][20]. Or real-time analytics can help an agent find the answer to a caller's question more quickly [8] or reduce the amount of time that agents spend documenting calls. We address the last goal in this paper.

The Contact Center Agent Buddies (CAB) project was undertaken jointly by IBM Research and a major automobile manufacturer (referred to henceforth as our "industrial partner") to develop and test new methods for improving contact center call handling and operations. The CAB project has two primary components: (1) a collection of Agent Buddies that "listen" to every phone conversation and, in real time, increase agent efficiency; and (2) a Transformational Diagnostic Tool (TDT), a background (offline) process that analyzes structured and unstructured data from the contact center. The goals of the CAB system are to reduce the average handle time (AHT) of calls and provide operational diagnosis. The project culminated in a "Proof of Concept" (PoC) in the industrial partner's contact center, to measure how well we did against the goals in a close-to-real business environment.

In this paper, we describe the part of the real-time system that aims to reduce the time spent handling customer calls by performing semi-automatic logging of the calls.

This paper is organized as follows. In the remainder of the introduction, we describe the agent-created call log and review previous work related to our system and underlying technologies. In Section 2, we present an overview of the CAB system in which the call logger is embedded, highlighting its dependency on other system components. In Section 3, we

describe the analysis pipeline and logger; in Section 4, we report and discuss the results of the PoC evaluation. We point toward future work in Section 5.

1.1 Contact Center Call Documentation

In a contact center, every call to or from an agent has to be documented, to support later continuation of the same customer interaction by the same agent, to support call transfer to another agent, or to support call evaluation by a supervisor. Agents may create documentation during the call or while the customer is on hold or immediately after completion of the call. Time spent in documentation is included in the call handle time. Many contact centers use a customer relationship management (CRM) system to maintain information about customer interactions, including call documentation. An agent typically takes notes in a text editor during the call and then transfers the information into a CRM record with cut-and-paste operations. At our industrial partner, a CRM record consists of a collection of fields, called a "template," containing data such as make, model, year, and VIN of the subject vehicle; owner information such as name, telephone number, and warranty status; and space for free-form call documentation (currently 1500 characters) that includes the caller's statement of the problem, relevant facts gathered from the caller (e.g. dealer, repair shop), and the agent's resolution. While this free-form documentation is indeed text, it is often quite irregular, characterized by idiosyncratic abbreviations, misspellings, missing words, grammatical errors, and incorrect case. The agent's goal is speed and completeness of documentation, not readability. This free-form documentation, together with the template fields, is what we term the call log.

1.2 Related Work

Automatic, or semi-automatic, call logging is an instance of dialogue summarization, with some specializations. There is already a body of research on summarization of speech of different genres: broadcast news, voice mail [5], meetings, and dialogue.

For broadcast news, approaches range from ones derived largely from text summarization[4] to strategies combining prosodic, lexical, and structural features[6] to exclusive reliance on acoustic/prosodic features.[7]

For meetings, Murray, et al.[9] demonstrate that summarization that takes into account features such as speaker activity, turn-taking, and discourse cues outperforms text-based methods inherited from text summarization.

For dialog summarization, Zechner [19] relies largely on tf/idf scores, tempered with cross-speaker information linking and question/answer detection. Presenting a project for summarizing dialogues in unrestricted domains, a pipeline architecture is described, with functionality distributed among different components. The article provides a comprehensive overview of the technical issues in summarization of dialogue—as opposed to summarization of, e.g., broadcast news—as well as a good overview of previous literature.

While call logging is similar to dialogue summarization, there are notable differences. The domain model for the call log, which describes what should be captured in the log, is partially embodied in a script that underlies at least part of the call (e.g. agent asking for phone number, whether the caller is the vehicle's original owner, what the VIN is). Unlike general open-domain dialogue summarization, which needs only to determine domain salience statistically, call log generation must attach semantic labels to domain information to route identified items into the correct fields of the CRM record.

In our project, we did not develop automatic methods for domain modeling, as, for example, Roy and Subramaniam[15]. Rather, we defined the domain model from our industrial partner's template structure and lists of vocabulary, such as vehicle makes and models, and UCC's (Uniform Commercial Codes).

2. CAB SYSTEM ARCHITECTURE

The goals of the Contact Center Agent Buddies project were to reduce the AHT in contact centers by improving the ability of the contact center agents to answer customer questions more rapidly, to reduce time spent in call documentation, and to improve first call resolution, another key contact center performance indicator.

The CAB system has three major subsystems: a speech subsystem, a real-time subsystem (Agent Buddies), and an agent/supervisor survey tool for evaluation of the real-time agent buddies.

The Contact-Center Agent Buddies system operates as follows. Every few seconds from the start of a call to the finish, it requests from its environment a speech transcript from the beginning to the current point in time. It then runs a (UIMA)[3] pipeline of annotators, or text analysis engines (TAE's)[1], over the transcript and updates its display to the agent based on any analysis changes since the last time it ran. The environment may be either a real-time production environment or a prerecorded simulated one.

The UIMA annotator pipeline takes the speech transcript and performs a variety of types of processing on it. Currently, the pipeline contains 40 annotators. Many of them perform basic text analysis functions commonly needed by all of the CABs, including those that we do not discuss in this paper. Others are more specialized. Basic common analysis involves, primarily, converting the speech stream into reasonably well-formed sentences or utterances, and then extracting the key task(s) that the agent must perform. This task is then evaluated by each of the Agent Buddies (CABs), which are specially designed components that take the task, evaluate it, and try to solve some aspect of it. For example, one CAB may create a log of the call, another may propose a particular document as a solution to a task, or yet another may try to perform operations that are time-consuming for agents, such as identification and data base lookup of vehicle identification numbers (VINs).

In order to reduce the integration complexity of the PoC, we did not feed live audio calls into the agent buddies system. Instead, we prerecorded three weeks worth of calls and generated the

[1] In UIMA, a TAE is a module for analyzing information about a document stored in a "common analysis structure" ("CAS") and for adding annotations to the CAS, representing new document information asserted by the TAEs.

ASR transcripts in one off-line pass. The call simulator provides the output of a pre-transcribed call to the main system polling the transcript in the same timing that the real-time transcription engine would. The call simulator decides, based on the timing information, how far along the call is and returns to the CAB system the transcript from the start to the current position. A live real-time environment component would generate a transcript on the fly and return its current state.

For its ASR component, our system uses a somewhat more recent version of the IBM Research "Attila" prototype speech transcription system described in [16]. The acoustic models used were trained on approximately 2000 hours of general conversational telephony speech data in addition to approximately 340 hours of data that was collected from the PoC call center. The language models used are a mixture of a general language model and a language model built from manual transcriptions of the same 340 hours of data. The general language model contains data from various sources: conversational telephony speech, broadcast news, etc. The ASR vocabulary is augmented with lists of automobile makes, models, and components obtained from the manufacturer. After applying various optimizations, the details of which are beyond the scope of this paper, we obtained raw transcripts which had a measured average word error rate (WER) of about 26%.

The agent/supervisor survey tool was specifically designed and implemented for use by the agents in the PoC and will be described later.

3. ANALYSIS PIPELINE AND LOGGER

Our approach to call log generation involves the data transformation steps shown at a high level in Figure 1.

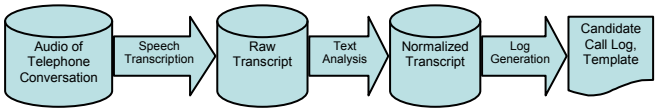

Figure 1. High Level View of Log Generation

Beginning with the audio of the conversation, we first use ASR to create a raw transcript of the conversation. Next, a pipeline of UIMA TAEs is used to create a normalized transcript that includes sentence boundaries. Finally, sentences are extracted from this normalized transcript to create the candidate call log, which is presented to the agent for editing and storage. In this section, we describe how these steps are performed.

3.1 Speech Transcription

The raw transcript resulting from speech transcription indicates only which words were spoken when by which speaker; there is no case information or punctuation in it. Tracking speaker turns is important for the Template and Call Logging CABs. The timing detail is more important for other CABs, which must present their results during an on-going phone call, than it is for the Template and Call Logging CABs, whose results may not be complete—or used—until the end of the call.

3.2 Token and Sentence Normalization

A suite of about twelve general-purpose UIMA text analysis engines (TAEs) "normalizes" the raw transcript on behalf of all the other TAEs and CABs in the system. In effect, the goal of normalization is to produce ordinary written text which a human reader would accept as readable. This is a strong requirement in the call logging application, since it should yield logs that can be easily comprehended when they are later used in other call center operations. We might suggest here that the human-created logs, littered as they are with misspellings and idiosyncratic abbreviations, do not provide too high a bar in comparison.

Normalization begins with removal of filler words (e.g., "uh") and interjections (e.g., "okay," when interrupting the other speaker). It continues with normalizing spoken numbers (e.g., "two thousand and seven" becomes "2007") and spoken spellings and acronyms (e.g., "j o n e s" becomes "jones"; "a b s" becomes "abs"). Furthermore, certain expression types, such as CRM record numbers or phone numbers, are transformed into their standard orthographic shapes (e.g., "123-456-7890" for U. S. phone numbers). Domain-specific words and acronyms are rendered with the expected capitalization patterns (e.g., "ABS"), as is the pronoun "I." Sentence boundaries are detected, using the methods described in [10], and the resulting sentences are given initial capitals and punctuated with periods.

It is important to note that these normalizations benefit all of the CABs, and not only the Call Logging and Template CABs. The normalization annotators are actually interleaved, as appropriate, with other UIMA annotators which perform more specialized analysis. Depending on the CAB, the normalized text is used either as a source of text for humans to read (especially true for the Call Logging CAB), as canonical values of domain-specific expressions (e.g., phone numbers or VINs for the Template CAB) to be stored in data repositories, or as parameters to other computing systems (e.g., a case or ticket number or a query to the solutions knowledge base).

A significant portion of the low-level text normalization work in the CAB system is performed by cascades of finite-state grammars written in the AFst (Annotation-Based Finite State Transducer) language.[1][2] Briefly, AFst supports a declarative notation for specifying patterns of annotations in a UIMA CAS and for describing creation, modification, and deletion operations on those annotations.

3.2.1 Disfluency Removal (Cleanup)

Specifically, cleanup and normalization of the raw speech transcripts begins with a set of FST grammars which removes words identified as disfluencies and interjections. A disfluency is a word such as "uh" or "hmm", which is in the ASR vocabulary. An interjection is an expression such as "yep", "oh wow", "great", etc., which is spoken by one speaker between two turns of the other speaker. When that occurs, the grammars also combine the speaker turns of the interrupted speaker into a single turn.

3.2.2 Expression and Acronym Normalization

More grammars are then applied to identify a range of expression types (henceforth *typed expressions*), including acronyms, alphanumerics (e.g. VINs) and various kinds of numbers, and to assign types to them (phone numbers, service request/ticket numbers, amounts of money, automobile mileage numbers, etc.). Expression typing most often requires

grammars that first create "cue" annotations over cue phrases, typically found in questions by the agent (e.g. "ticket number," "VIN number," "17-digit number," "phone number") that are followed by caller turns containing the information of the type corresponding to the cue.

Note that alphanumeric expressions are often used in designating automobile models and trim levels (for example "R8", "328xi"). An associated non-grammar-based TAE is used to compute the best normalized value for these expressions. For acronyms it concatenates the spelled initials into a single token; for numbers, it computes the standard decimal representation.

3.2.3 Case Normalization

As the raw transcript contains no case information about the words it contains, we built a TAE whose job is to determine which words should be upper-cased, leaving everything else in lower-case. It begins simply by upper-casing all tokens based on the pronoun "I" (e.g., "I'm"). It then uses annotations of domain-specific words and phrases that have been put into the CAS by a prior application of a dictionary look-up TAE. In our case, the dictionary was created from lists obtained from the industrial partner: automobile makes, models, trims, and components, as well as their variants, acronyms, and abbreviations. Whenever the case normalization TAE finds that a variant of a domain term has occurred in the transcript, a case pattern is derived from the term's lexical entry. This allows us to produce standard spellings for expressions such as "ABS", "Mercedes-Benz", or "TrailBlazer", for example.

Note that capitalization of the first word of a sentence is not done at this time. That must wait for the next step, which is sentence boundary detection.

3.2.4 Sentence and Segment Boundary Detection

An utterance boundary detector[10] divides the continuous stream of words from the recognition engine into normal sentences. The algorithm, based on Maximum Entropy classification, uses linguistic and prosodic features such as the probabilities of unigrams and bi-grams occurring as the first or last unigram (or bi-gram) in utterances, and the length of pauses between two words. Once sentences have been identified automatically, we capitalize the first words of the resulting sentences and add a sentence-ending punctuation mark to each.

Next, analysis of call transcripts reveals that calls can be divided into segments that denote progress through a standard call flow. We work with these twelve domain-independent segment types:

- Greeting
- Customer Information
- Question
- Refinement
- Research
- Resolution
- Follow-up Scheduling
- Agent Information
- Service Request Information
- Closing
- Transfer (usually happens at beginning or end of call)
- Out-of-topic (usually during Resolution)

In addition, for the automotive domain, we identify these domain-specific call segment types:

- Dealer Information
- Vehicle Information
- VIN Identification

Several of our system's processes, including the Call Logging CAB, require knowledge of which segment type contains a portion of the transcript that they specialize for. After the utterances are identified, call section segmentation (see [10]) applies a Support Vector Machine (SVM) classifier trained on manually segmented calls to assign one of the target call segment types to each utterance. Adjacent utterances of the same section type are merged.

3.3 Template Generation

The Template CAB extracts the information required for the different structured fields in the CRM record (template). This information is identified in one of two ways. In the first case, a manually created domain dictionary or semantic lexicon drives a dictionary annotator to create annotations specifying semantic types (e.g. vehicle brands and models). The second method uses extraction patterns encoded as manually written finite state grammars. This method is most often used to extract non-vocabulary artifacts such as telephone number, vehicle year, warranty status, or VIN.

3.4 Log Generation

The job of the Call Logging CAB is to prepare a candidate call log that is presented to the agent for editing and for storing in the Comments field in the CRM activity record that is created for the call. It creates the candidate log by selecting sentences from the normalized call transcript that appear to contain information that best characterizes what happened during the call. A number of heuristics, based on expression content, lexical content, and syntactic analysis, are used to select these sentences and to assign importance scores to them. Further syntactic analysis is used to label the selected sentences which belong to the log sections that describe the customer statements, the customer request, and the next action/solution advised by the agent. After labeling, a number of sentences sufficient to create a log record of the required length is extracted from the normalized transcript and copied into the "automatic log editor".

This processing begins with two cascades of finite state grammars. The first cascade creates "cue" annotations over the transcript, indicating locations where material important for a call log might be found. The cue types annotated are:

- Advice ("What I can do for you is …")
- Apology ("I'm sorry to hear about your trouble …")
- Concern (where the user expresses a vehicle complaint or request. Note that the grammar also creates a Concern for any Symptom annotated by dictionary annotator using a symptom dictionary.)
- HoldTime ("Could you hold for 3 to 5 minutes?")
- ReportedSpeech ("The dealer told me that …")
- SeekingQuestion ("I wanna see if …")
- SoughtItem ("reimbursement")
- AnythingElseQuestion ("Is there anything else I can help you with today?")

The second set of preliminary grammars uses the cues and other syntactic structures to determine when a sentence should be annotated with one of three "candidate" annotation types. The types are:

- CustStates (i.e., the caller describes the problem)
- CustSeeks (i.e., the caller asks for something specific)
- CrmAdvises (i.e., the agent offers to take some action)

After these preliminary annotations have been added to the transcript, the call log generator assigns a score to each sentence. Currently, the score for a sentence is a simple count of the number of typed expressions, domain terms, and log cues found in the sentence. Although a more sophisticated empirical approach to computing these scores may eventually be preferable, we have not found it necessary in our work so far. In particular, while normalizing the scores for sentence length would ordinarily be advisable to reduce the bias toward long sentences, we have not done so because (a) we prefer longer sentences for our logs and (b) our sentence boundary detection heuristic tends not to produce very long sentences, anyway.

Note that we omit some of the typed expressions from the score assigned to a sentence. Specifically, those expressions which the Template CAB will present to the user either in the CRM record or in the structured part of the log (see the discussion below under "Business Results") should not be redundantly given in the text of the log. The expression types we ignore include phone numbers, ticket numbers, and VINs.

We build the candidate log text by extracting enough of the highest-scoring candidate-annotated sentences to create a log record that fits into the space available for it in the CRM record. The selected sentences are added to the candidate log in the order in which they occur in the transcript. Each sentence (or group of adjacent sentences) is marked with a label (e.g., "Cust States:"), based on the type of its candidate annotation. A final cleanup is performed if the log record begins with a "CRM Advises" label. In that case, we try harder to find a (lower-scoring) sentence that is a CustStates or CustSeeks candidate and that precedes the first CrmAdvises sentence.

An anonymized sample candidate log record is shown in Figure 2. Note that, at the top of the record, we have added the Year, Make, and Model of the customer's vehicle. This information would have appeared in sentences or sentence fragments spoken by the caller somewhere in the call and would have been annotated as domain terms, but, because they appear in the template as identified by the Template CAB, their sentences were not selected for the log. However, during the PoC testing, the agents requested this display. Ellipses within a paragraph indicate that the surrounding sentences were not adjacent in the normalized transcript. Finally, we save the agents some typing by pre-adding a "signature line", containing the agent's name, organization, and work location.

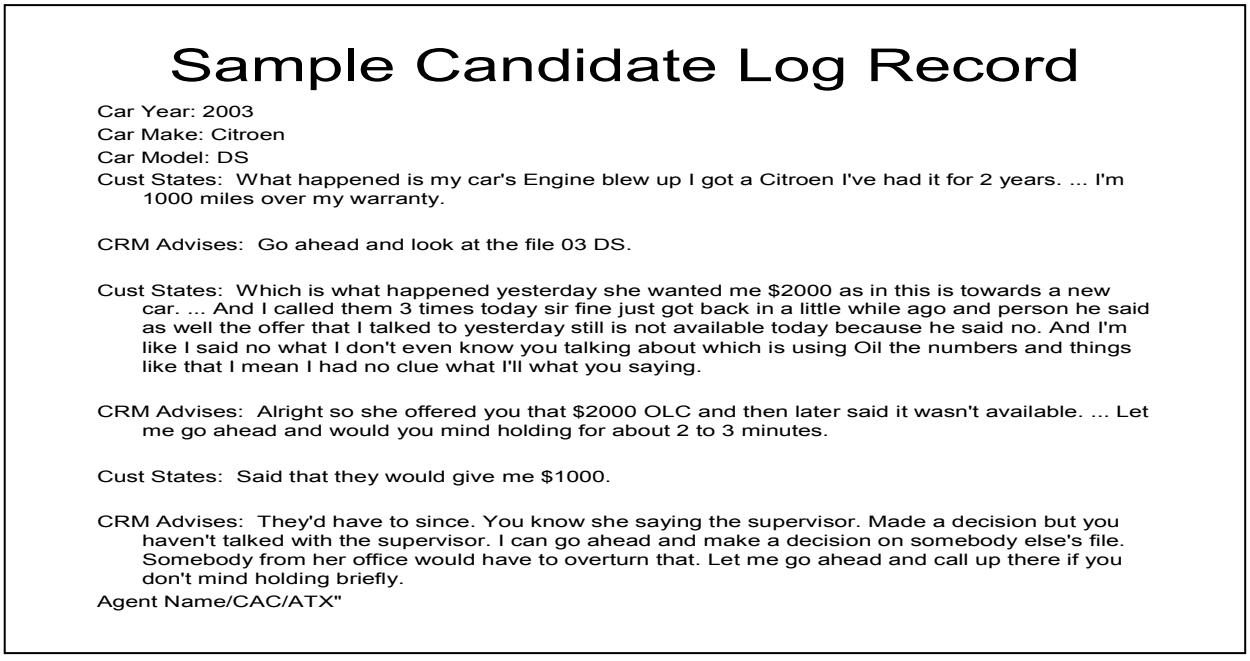

Figure 2. Sample Candidate Log Record

4. BUSINESS RESULTS

One of the goals for the CAB project was to show our industrial partner that, by using CABs, contact center agents could save substantial amounts of time during call handling. To do this, we designed a Proof of Concept (PoC) test that allowed us to measure actual agents' use of the integrated CAB system with transcripts generated from recordings of previously-handled calls. The test included a supervisor survey tool that allowed us to get contact center supervisors' evaluations of the agents' performance and quality of results when using the system.

We conducted a two-week PoC in one of the several contact centers where our industrial partner's customer support calls are actually handled. For the test, we selected eight agents who normally provide support for some of the partner's car brands. Four of the agents had fewer than six months tenure in that support center; the other four had more than six months tenure. We also used two supervisors from the same support center. One of them had been in this position only two months; the other was much more experienced.

4.1 Data Description and Statistics

Calls used during the PoC were recorded during the three weeks preceding the start of the test. All of the calls were about the manufacturer's vehicles in North America and could have been handled originally at any of the contact centers, not just the one where the PoC was conducted. Beginning with 3775 audio files of "phone transactions," we applied filters to select inbound calls only. We also eliminated calls of less than two minutes and more than ten minutes of talking time. This was to ensure a minimum amount of content, to avoid quick transfer calls, and to limit the time spent on calls during the test. Calls that contained no mention of a brand or model handled in the PoC (we focused on a smaller subset of the brands) were also excluded. The set that passed all the filters was used as the basis for the PoC. It contained a total of 646 calls.

Each of these calls was assigned to both an experienced and a less experienced agent to evaluate the usefulness of the CAB system for these two groups. As our automatic filters were not perfect, we asked the agents and supervisors to manually exclude (i.e., skip) these same categories and several additional ones that could not be excluded automatically from the analysis:

1. Outbound calls
2. Phonemail messages
3. Calls about brands not supported by this group of survey agents
4. Transfers to agents in another support center;
5. Defective sound files
6. Disconnected calls
7. Run-time errors
8. Calls in a language other than English
9. Calls originally handled by the survey agent himself/herself

4.2 Survey Administration

On each day of the PoC, the agents each participated in a two-hour session during which they listened to the recorded calls through individual headphones and reacted to the results of the various CABs presented in a survey tool on their computers. The agents had a notepad application available to take notes during the call. While listening to the call, each agent was asked to rate the results of the CABs, to create their own log of the call, and to edit and then rate the automatically-created candidate log from the Call Logging CAB. The amount of time agents spent taking notes and creating their own log was captured. The agents also filled out an end-of-call survey and an end-of-day survey to assess their reactions to the various parts of the project. The results of this activity were captured and stored by the survey tool.

Supervisors spent three hours each day listening to calls from the same collection, assessing the quality of the call handling and the call logs, and using other tools which will not be discussed in this paper.

If an agent or supervisor detected that any call was outside of the scope of the PoC (i.e., if it failed to pass the filters mentioned earlier), they immediately clicked the "Skip" button which marked that call so that other agents would not need to listen to it.

In general, our survey subjects were given no special instructions, except that, on the first day of the PoC, we walked them through all parts of the survey tool to ensure that they understood the tasks. Participants were encouraged to work at their normal speed.

The survey was used to evaluate performance improvement for all of the CABs in our system. In the remainder of this paper, however, we restrict our attention to measurements concerning the Call Logging CAB.

4.3 Results

Using data obtained during the PoC, we wanted to establish two things about the logs from the Call Logging CAB. First, we wanted to show that, using the CAB, agents can create logs faster than they do today using NotePad, Microsoft Word, and/or the CRM's text editor. Second, we wanted to show that this increase in speed does not lead to a reduction in the quality of the logs that are created.

The Call Logging CAB was evaluated by having the survey agents create two logs for each call that they listened to. The first one emulated the normal log-writing conditions experienced by the original agents when they took the call. That is, the survey agents were given a NotePad-like editor in which they made whatever notes they wanted while listening to the call. At the end of the call, they used these notes within a manual-log-editing window to create their manual log. (All of the survey tool's editing windows were instrumented so that the time agents spent using them could be measured and recorded in the agent evaluation data.) Time spent in both the NotePad editor and the log editor was measured and summed to give the "manual log creation" time. The second log for each call was created by editing a candidate log generated automatically by the Call Logging CAB, within an automatic-log-editing window. During the test, we collected a dataset comprising 275 survey agent evaluations of 179 calls. (Recall that calls were each evaluated by two agents.) These evaluations contained 275 manual logs and 269 edited automatic logs.

It should be stressed that, for our evaluation, our baseline was the manual log creation time of the survey agents and not of the agents who originally handled the calls we recorded. Since we could not instrument the operational system used by the original agents, we did not have access to information about the time they spent writing their own logs for those calls. We believe, however, that the survey agents' manual log creation times constitute a more "conservative" baseline for our measurements, since they are, in general, shorter than those of the original agents. One reason for this is that the original agents often wrote log material while the customer was placed on hold, whereas the survey agents did not.

4.3.1 Time Savings

We measured daily averages for manual log creation time and automatic log editing time, over the course of the first nine days of the PoC. As shown in Figure 3, we observed that, whereas average manual log creation time remained more or less the same throughout the test (ranging from 250 to 300 seconds per call), automatic log editing time decreased from an average of 225 seconds per call on the first day to 100 seconds per call on the ninth day).

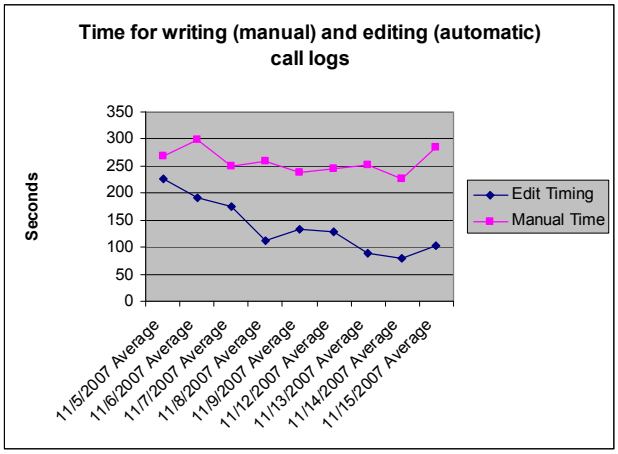

Figure 3. Time for writing (manual) and editing (automatic) call logs

We believe that the increased savings result from increased familiarity of the survey agents with the style and properties of the automatically-generated logs. Figure 3 shows that, over the same time period, there was no significant change in the length of time it took the survey agents to write their manually-generated logs. It remained at between 250 and 300 seconds for the whole PoC. However the automatically-generated logs were created steadily faster. Furthermore, several of the agents told us that they thought that the quality of the automatically-generated logs had improved during the course of the experiment. In truth, nothing changed beyond the aforementioned addition of the Template CAB results; certainly nothing in the quality of the ASR or the text normalization improved during the test. The only explanation for the agents' opinion is that they became more familiar and facile with the automatically-generated logs over time, and perhaps more tolerant of certain types of ASR errors.

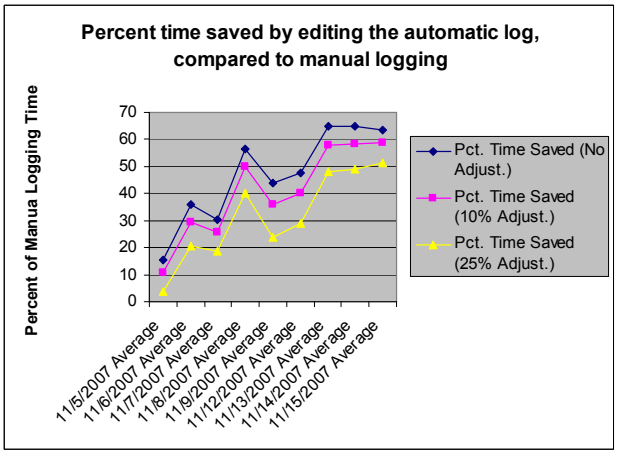

Figure 4. 50% to 65% of log-creation time can be saved by editing the automatic log, compared to creating a manual log

Using these timing results, we estimated the proportion of time a trained agent can save using the Call Logging CAB. Figure 4 shows that, by the final day of our measurements, agents editing the automatic logs were using only one-half to one-third of the time required to log the call manually. The range of estimates here is due to our varying estimates of how much of the time the survey agents spent writing in NotePad contributed to creating the final edited automatic log. The curves in the graph show a final savings of 63%, 58%, or 51%, corresponding to estimates that the adjustment should be 0%, 10%, or 25%, respectively. The actual amount here will depend on the GUI; if the system is able to give early feedback about the extracted facts, agents may not find it necessary to take as many notes.

Based on these measurements and estimates, we conservatively claim that the Call Logging CAB will result in at least a 50% reduction in the amount of time agents spend logging their calls, once they become familiar with the tool. This is conservative because it is based on the assumption that 25% of the NotePad contents are used in the edited automatic logs – an amount that far exceeds the proportion we observed in the actual logs created by the survey agents.

4.3.2 Log Quality

Our second evaluation objective was to show that any speedup we experience with the Call Logging CAB does not come at the expense of logging quality. To show that, we asked the supervisors to evaluate the quality of the logs by assessing the extent to which a number of quality statements applied to them. Our measurements showed that the supervisors judged the edited automatic logs to be better than logs written by the original agents, as well as those written by the survey agents.

After listening to each call, the supervisors assessed up to four logs:

- The log generated by the original agent who took the call – 39 total
- The log generated by the survey agent – 61 total
- The unedited automatically-generated log from the Call Logging CAB – 60 total
- The edited log from the Call Logging CAB, edited by the survey agent – 44 total

These four logs were presented to the supervisors in random order, in order to avoid any bias. It was apparent, however, that the supervisors could tell the difference between the human-written logs and the automatically-generated ones. One difference, for example, was that the automatically-generated ones had more consistent capitalization patterns, due to our text normalization.

The quality statements assessed were:

- "The log adequately captures important parts of the call."
- "The log accurately captures what was actually said in this call."
- "The next agent taking a call from this same customer would consider this log: [Essential / Nice but not necessary / Not particularly useful]"
- "In terms of detail, this log is: [Too detailed / About right / Not detailed enough]"

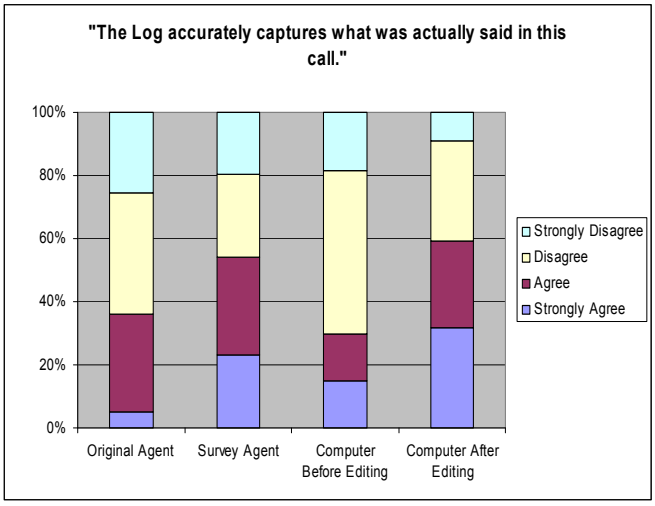

Figure 5. Supervisor evaluations of log accuracy

The assessments by the supervisors showed us that, in general, they preferred the edited automatically-generated logs to the manual logs generated both by the original agents and by the survey agents. Figure 5 shows this for the question "The log accurately captures what was actually said in this call." Supervisors strongly agreed with this statement over 30% of the time for the edited logs, while they strongly agreed with it less for both types of manual logs. The sum of "agree" and "strongly agree" judgments (almost 60%) for edited logs also exceed that for either of the manual log types.

Similarly, Figure 6 shows supervisor assessments of log detail. They thought that the edited logs were "about right" more frequently than for the manual logs. And the proportion of calls that they thought were "not detailed enough" was lower for the edited automatic logs. Note that the supervisors also thought that some of the automatically-generated logs were "too detailed." This may not be surprising, since, in general, the CAB's logs were longer than the manual ones.

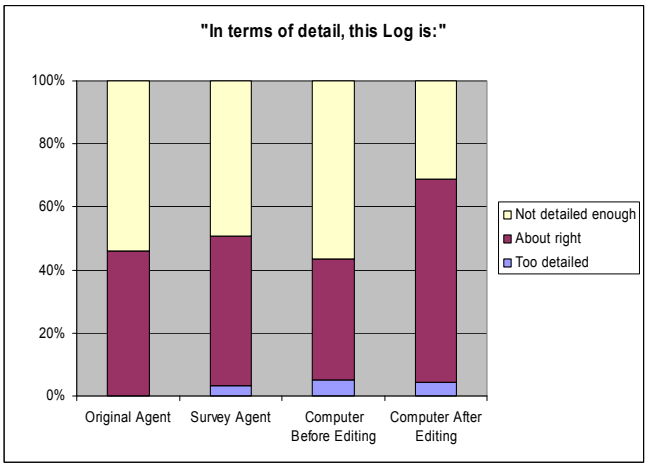

Figure 6. Supervisor assessment of log detail

The data for the other two statements is not shown here, but is consistent with the claim that the quality of the logs generated by the Call Logging CAB is not lower than the quality of manually-generated ones. This accomplishes our second objective.

4.4 Discussion

The quality statements we asked the supervisors to assess were motivated by the function that logs serve in the contact center environment. Logs are required to be accurate and complete records of what was said during customer calls, to support future use both by the original agent and by a new agent who may take up the case. In addition, they are used by supervisors when doing agent performance evaluations. Even though our measurements established that, using the Call Logging CAB, agents can generate log records that meet these requirements at (conservatively) half the cost, we might nevertheless ask ourselves how it can be that imperfect speech recognition and text analysis technology can succeed.

We believe that part of the answer lies in the mechanics of the Call Logging CAB. The normalization that it performs yields text that may be more readable because it contains correctly spelled words and consistent capitalization and punctuation. This contrasts with manual logs which are frequently quite irregular. Next, the 26% WER (word error rate) of our ASR tends to be due more to smaller words (e.g., function words) and less to domain terms. Therefore, word errors often do not affect comprehension. And when an error must be corrected, it may be easier for the agent to correct it than it would have been to capture the whole thought manually.

To these observations, we add that automatically-generated logs may be more consistent than manually-generated ones. Automatic paragraph labeling is a reliable indicator of who said what during the call. The management and use of domain terminology allows the Call Logging CAB to use consistent spelling of domain terms such as names of components, models, and trim levels – things which vary considerably in manual logs. The fact that everything said during a call is available for selection by the log generation heuristics decreases the

likelihood that something important will have been missed by the agent in a manual log.

5. FUTURE WORK

When an agent edits an automatically-generated call log sentence, it is possible to distinguish changes which correct speech recognition errors from changes made for other reasons. Using methods akin to those described in Yu, et al.[18], the speech corrections could be used as feedback to the ASR subsystem, in order to improve its performance and so reduce editing time. Specifically, lexical corrections can be used to improve the lexicon in the ASR system's language model. Not only will the agent have fewer word or phrase errors to fix, the sentence selection mechanism will make better choices due to fewer errors in domain vocabulary. We plan to experiment with these approaches, as one of our techniques for enhancing the domain adaptability of the CAB system.

The accuracy of VIN capture (as well as that of alphanumeric domain artifacts in general) suffers more than other extracted types from ASR errors. In contexts identified by extraction patterns, annotator access to the ASR confusion matrix (not available downstream from the ASR in the current implementation), can improve precision and recall of numeric and alphanumeric data (cf. Rahim et al. [11][12]).

Having demonstrated that semi-automatic call logging can save a considerable amount of time with no loss in quality, we are studying two different directions in usability: adapting to other domains, and exploiting the possibilities of real time.

The question of domain adaptation is, how we can easily port the system to a different auto manufacturer with different logging requirements or to a different industry. Because the system described here was implemented quickly, it was expedient to install into it the known domain model by finding out what our industrial partner deemed important to keep in the CRM record. We chose to build dictionaries of domain terms with their semantic labels from lists rather than to discover them automatically. We wrote finite state grammars to search for patterns containing particular cues, specified distance between cue and item as a variable window size, and specified the semantic type of the desired item for the template. Other systems—without the requirement for attaching semantic labels to extracted data—are more easily portable.

For porting to another manufacturer or industry, we would, at a minimum, parameterize (from the outside) cue types, grammars, Template CAB, and Logging CAB. For the Template CAB, we are currently considering an approach along the lines suggested by [13], [14] that can learn domain vocabulary and extraction patterns for the new manufacturer or industry. A promising source of training data is existing transcripts in which the data found in the matching CRM records is annotated. Due to a number of difficulties, however, the volume of transcripts that we can accurately match with CRM records is not sufficiently large. Or, more generally, we would study ways to endow an already existing domain modeling strategy with the ability to deliver the required typing information needed to determine the semantic roles filled by the domain terminology and expressions.

Prospects of further reducing the call handling time, e.g. with the Template CAB, open up when the Agent Buddies are running in real time, rather than in our simulated test system. We will mention just one. Analysis of the time spent in different parts of the conversation showed large amounts of time spent in transmission of the VIN from the caller to the agent. While we estimated in the PoC that we could save agents some time in the activities of typing the VIN, context switching for data base look up, looking it up, and copying the VIN to the template field, we determined that a significant time saving possibility lay elsewhere: in the human factors of getting an uninitiated caller to understand what a VIN is, where to find it (vehicle registration sticker on the windshield), and how to read it without tripping, making errors, or restarting. (Recall also that the VIN, as a 17-digit alphanumeric, is most vulnerable to both human errors and ASR errors.) One solution would be for the Automatic Voice Recognition System (AVR) that first picks up the call to tell callers that it wants the VIN now. Recitation to a known machine is probably cleaner than reading to a human (less likely for the caller to say, "three two I guess its an eight"). Or, if agents know the characteristics and strengths of the Agent Buddy to recognize the information that the caller is transmitting, they can guide the caller to do the optimal thing for recognition and analysis (for example, interrupting a long VIN recitation with nothing more than a reassuring "yep") or explain that the caller is talking to a machine. Fewer different caller behavior patterns would have to be supported by the CABs, because the agent is getting instant feedback in the displayed template of elicitation failure and can guide a retry. Variations for other data types can also be imagined.

6. ACKNOWLEDGEMENTS

We are grateful to Larry Bates for alerting us to the importance of call logging in the contact center environment and for encouraging us to focus on AHT reduction as a success metric for our work. In any system-building effort in a company like IBM, there are many people behind the scenes. We thank the IBM First-of-A-Kind (FOAK) team for helping us get initial funding for the project and the IBM Managed Business Process Services (MBPS) organization for their support of interactions with our industrial partner. We are grateful to many unnamed people at our industrial partner for answering countless questions about how their operations work. We especially want to acknowledge the expertise and generosity of the agents, supervisors, and management at the PoC contact center. Finally, we thank Branimir Boguraev, co-inventor of AFst, for his advice and support as we wrote our analysis grammars.

7. REFERENCES

[1] Boguraev, B. and Neff, M. 2008. Navigating through Dense Annotation Spaces. In Proceedings of the Sixth International Language Resources and Evaluation (LREC'08), Marrakech, Morocco, May, 2008.

[2] Boguraev, B. and Neff, M. 2003. The Talent 5.1 TFst System: User Documentation and Grammar Writing Manual. IBM Technical Report RC 22976. Update to appear, 2008.

[3] Ferrucci, D. and Lally, A. 2004. UIMA: An architectural approach to unstructured information processing in the corporate research environment. In Natural Language Engineering, 10(3):476-489.

[4] Hori, C., Furui, S., Malkin, R., Yu, H., and Waibel, A. 2003. A Statistical Approach to Automatic Speech Summarization. EURASIP Journal on Applied Signal Processing 2003:2, 128-239.

[5] Koumpis, K. and Renals, S. 2005. Automatic Summarization of Voicemail Messages Using Lexical and Prosodic Features. In ACM Transactions on Speech and Language Processing, Vol. 2, No. 1, February 2005, Article 1.

[6] Maskey, S. and Hirshberg, J. 2005. Comparing lexical, acoustic/prosodic, discourse and structural features for speech summarization. In Proceedings of the 9th European Conference on Speech Communication and Technology, Lisbon, Portugal, September, 2005.

[7] Maskey, S. and Hirshberg, J. 2006. Summarizing Speech Without Text Using Hidden Markov Models. In Proceedings of the Human Language Technology Conference of the North American Chapter of the ACL, New York, June 2006, 89-92.

[8] Mishne, G., Carmel, D., Hoory, R., Roytman, A., and Soffer, A. 2005. Automatic Analysis of Call-center Conversations. CIKM'05 (Bremen, Germany), 453-459.

[9] Murray, G., Renals, S., Carletta, J. and Moore, J. 2006. Incorporating Speaker and Discourse Features into Speech Summarization. In Proceedings of the Human Language Technology Conference of the North American Chapter of the ACL, New York, June 2006.

[10] Park, Y. 2007. Automatic Call Section Segmentation for Contact-Center Calls. CIKM '07 (Lisboa, Portugal, November 06-08, 2007), 117-125.

[11] Rahim, M., Riccardi, G., Saul, L., Wright, J., Buntschuh, B., and Gorin, A. 2001. Robust numeric recognition in spoken language dialogue. In Speech Communication 34 (2001) 195-212.

[12] Rahim, M., Riccardi, G., Saul, L., Wright, J., Buntschuh, B., and Gorin, A. 2007. Recognizing the numeric language in natural spoken dialogue. United States Patent 7,181,399, February 20, 2007.

[13] Riloff, E. 1996. Automatically Generating Extraction Patterns from Untagged Text. In Proceedings of the AAI-96.

[14] Riloff, E., and Wiebe, J. 2003. Learning Extraction Patterns for Subjective Expressions. In Proceedings of the 2003 Conference on Empirical Methods in Natural Language Processing – Volume 10, 105-112.

[15] Roy, S. and Subramaniam, L. 2006. Automatic Generation of Domain Models for Call Centers from Noisy Transcriptions. Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL (Sydney, 2006), 737-744.

[16] Soltau, H., Kingsbury, B., and Zweig, G. 2004. The IBM 2004 Conversational Telephony System for Rich Transcription. In Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2004.

[17] Takeuchi, H., Subramaniam, L., Nasukawa, T., Roy, S., and Balakrishnan, S. 2007. A Conversation-Mining System for Gathering Insights to Improve Agent Productivity. E-Commerce Technology and the 4th IEEE International Conference on Enterprise Computing, E-Commerce, and E-Services, 2007, CEC/EEE 2007, Tokyo, Japan.

[18] Yu, D., Hwang, M., Acero, A., and Deng. L. 2004. Unsupervised Learning from User's Error Correction in Speech Dictation. In INTERSPEECH-2004, October 2004, 1969-1972.

[19] Zechner, K. 2002. Automatic Summarization of Open-Domain Multiparty Dialogues in Diverse Genres. Computational Linguistics, December 2002, 447-486.

[20] Zweig, G., Siohan, O., Saon, G., Ramabhadran, B., Povey, D., Mangu, L. and Kingsbury, B. 2006. Automated quality monitoring for call centers using speech and NLP technologies. Proceedings of the Human Language Technology Conference of the NAACL, Companion Volume, 292-295, New York City, June, 2006.